Software Requirements Specification

For Askify - PDF Question-Answering Service

Version 1.0
Prepared by: Shaik. Shareef
Organization: 6AI15, CSE-AI, B.Tech, Parul University
Date: 24-02-2025

Table of Contents

1. **Introduction**
    - 1.1 Purpose
    - 1.2 Document Conventions
    - 1.3 Intended Audience and Reading Suggestions
    - 1.4 Product Scope
    - 1.5 References

2. **Overall Description**
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
    - 2.6 User Documentation
    - 2.7 Assumptions and Dependencies

3. **External Interface Requirements**
    - 3.1 User Interfaces
    - 3.2 Hardware Interfaces
    - 3.3 Software Interfaces
    - 3.4 Communications Interfaces

4. **System Features**
    - 4.1 PDF Uploading
    - 4.2 Question-Answering System
    - 4.3 Search and Retrieval

5. **Other Nonfunctional Requirements**
    - 5.1 Performance Requirements
    - 5.2 Security Requirements
    - 5.3 Software Quality Attributes

6. **Other Requirements**

7. **Appendices**

---

1. Introduction

1.1 Purpose

Askify is a backend service that enables users to upload PDF documents and retrieve answers based on the content of those documents. The system leverages **LangChain** for natural language processing and **ChromaDB** for efficient document retrieval.

1.2 Document Conventions

- Headings are numbered for clarity.
- Important terms are **bolded**.
- Code snippets are presented in *monospace font*.

1.3 Intended Audience and Reading Suggestions

- **Developers:** Focus on Sections 2, 3, and 4 for system functionality and API details.
- **Project Managers:** Refer to Section 1 for an overview and Section 5 for requirements.
- **Users/Testers:** Review Sections 3 and 4 for system interfaces and functionalities.

1.4 Product Scope

Askify is designed to:

- Allow users to **upload PDFs** and extract text.
- Answer user queries based on uploaded documents using **retrieval-augmented generation (RAG)**.
- Store and retrieve document embeddings efficiently using **ChromaDB**.

1.5 References

- FastAPI documentation: https://fastapi.tiangolo.com/
- SQLite documentation: https://sqlite.org/
- LangChain documentation: https://www.langchain.com/docs

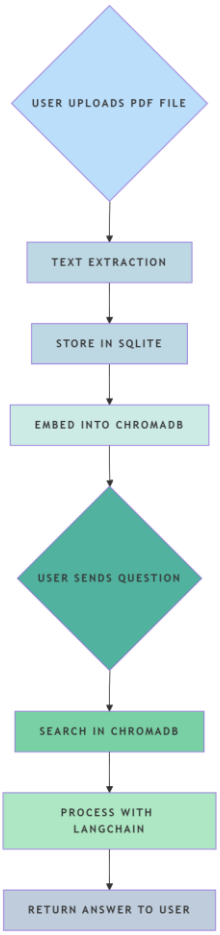
2. Overall Description

2.1 Product Perspective

Askify is a **new product** designed to facilitate document-based question answering. It integrates with **FastAPI** as the backend framework and uses **WebSockets** for real-time communication.

2.2 Product Functions

- **PDF Upload:** Users upload PDFs for processing.
- **Text Extraction:** Extracted text is stored in **SQLite**.
- **Question-Answering:** Users ask questions, and the system retrieves the most relevant answer.
- **Flowchart 1: System Architecture:**



2.3 User Classes and Characteristics

- **Students & Researchers:** Upload and query academic PDFs.
- **Legal & Business Professionals:** Extract key insights from contracts or reports.

2.4 Operating Environment

- **Backend:** Python 3.8+, FastAPI, SQLite, ChromaDB.
- **Hosting:** Local machine or cloud deployment (AWS/GCP).

2.5 Design and Implementation Constraints

- **Data Size:** PDFs must be **under 50MB**.
- **Concurrency:** Limited simultaneous WebSocket connections.

2.6 User Documentation

User manuals will be provided in **Markdown** and **PDF formats**.

2.7 Assumptions and Dependencies

- Requires **internet connectivity** for API-based NLP processing.
- API keys for **LangChain and Google AI** must be set up.

---

3. External Interface Requirements

3.1 User Interfaces

- API-based, tested using **Postman** and **WebSockets**.
- User interacts via a **simple web client or CLI**.

3.2 Hardware Interfaces

- Cloud-based database and **local storage** for document processing.

3.3 Software Interfaces

- **FastAPI backend**
- **SQLite database** for text storage
- **LangChain API** for NLP processing

3.4 Communications Interfaces

- **WebSockets** for real-time chat-based interactions.
- **REST API** for PDF uploads and document retrieval.

---

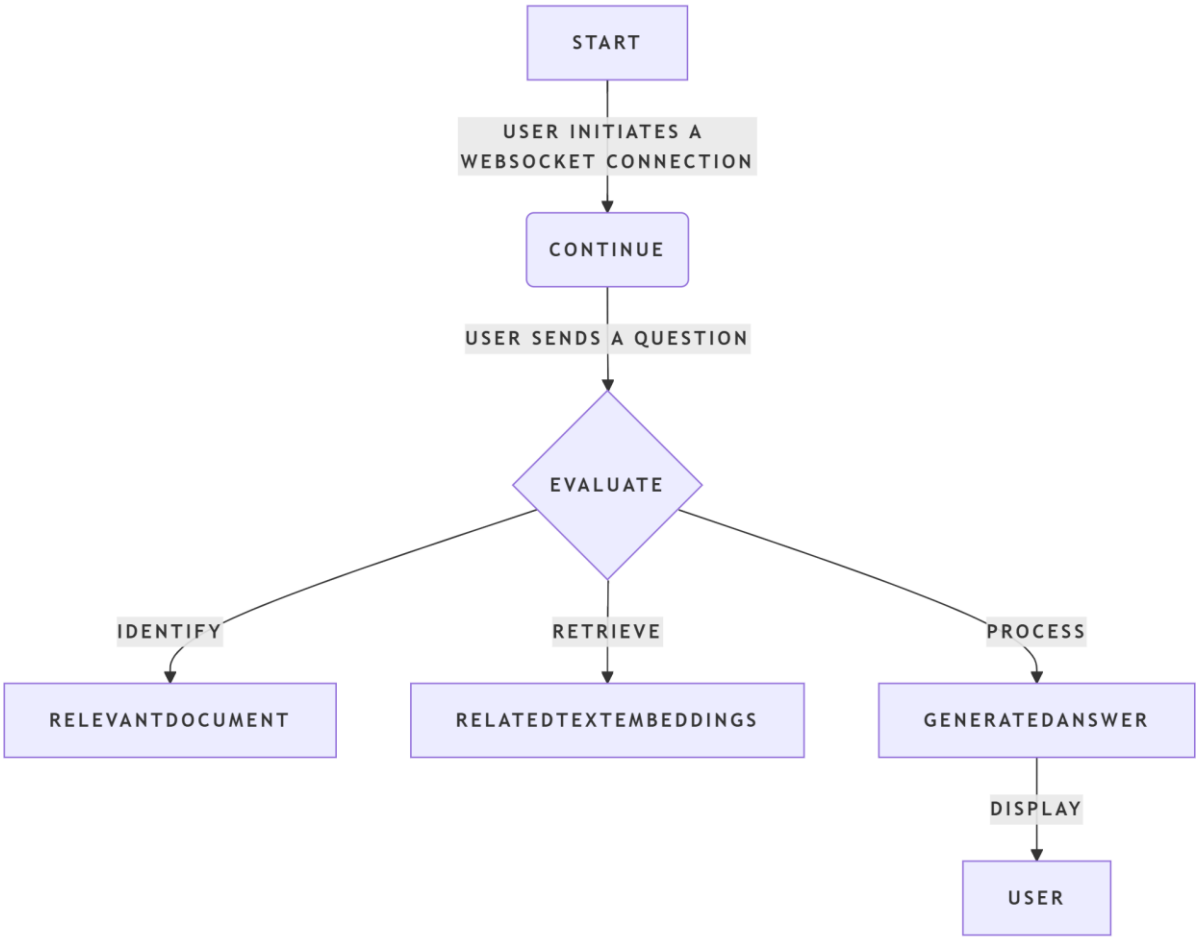
4. System Features

4.1 PDF Uploading

- Users can upload PDFs via POST /upload.
- PDFs are processed and stored securely.

4.2 Question-Answering System

- Users send queries via **WebSocket connections**.
- The system retrieves relevant text passages and responds in real-time.
- **Flowchart 2: Question-Answering Process**



4.3 Search and Retrieval

- Uses **ChromaDB embeddings** to improve search accuracy.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Response time for a **query should be <2 seconds**.
- Supports up to **100 concurrent users**.

5.2 Security Requirements

- **Authentication:** API keys required.
- **Data Encryption:** Documents are stored securely.

5.3 Software Quality Attributes

- **Scalability:** Cloud-hosted with **horizontal scaling**.
- **Maintainability:** Modular design with **well-documented code**.

---

6. Other Requirements

- Future support for **OCR-based PDF processing**.

---

7. Appendix A: Glossary

- **FastAPI:** A Python framework for building APIs.
- **LangChain:** An AI framework for NLP applications.
- **ChromaDB:** A vector database for text embeddings.
